Neural Networks II

Machine Learning 1 — Lecture 7

5th May 2026

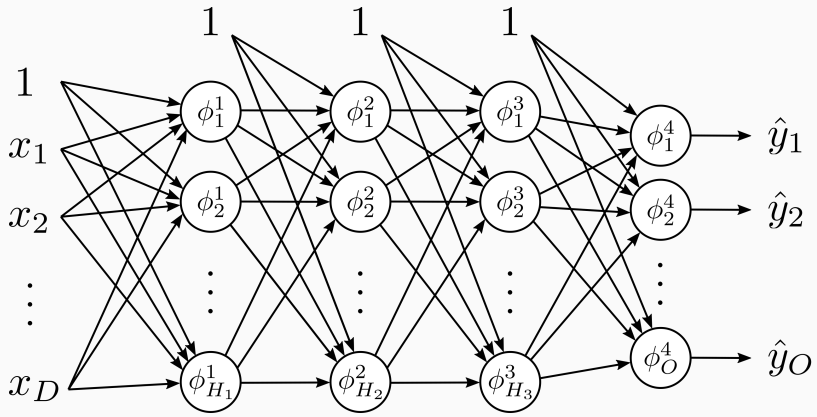
Robert Peharz
Institute of Machine Learning and Neural Computation
Graz University of Technology

Neuron, Unit

The **neuron** (**unit**) is the basic building block of neural networks:

$$\hat{y} = \phi(a) = \phi\left(\sum_{i=0}^{D} w_i x_i\right)$$

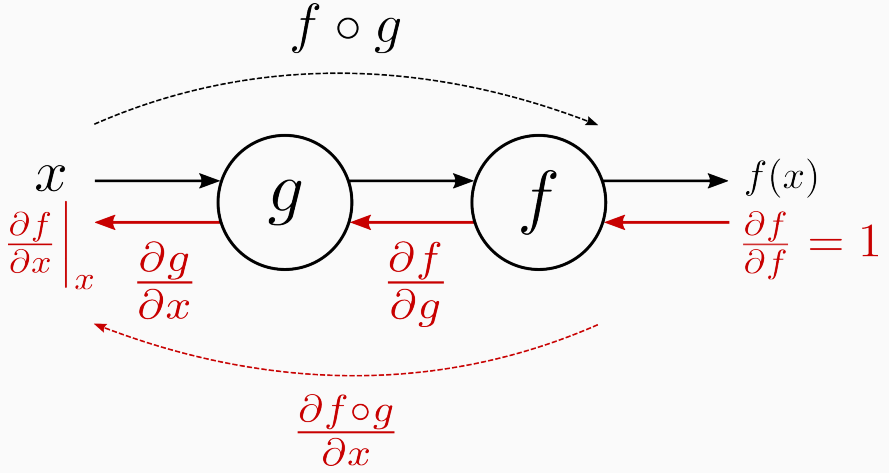
- D: number of inputs
- x_i: inputs (features, outputs from previous neurons)
- w_i: weights
- w_0: bias term ($x_0 \equiv 1$)
- ϕ: non-linear **activation function** (also called **non-linearity**)
- a: **activation** $\sum_{i=0}^{D} w_i x_i$
- $\hat{y}$: output $\phi(a)$
- **affine function of x, followed by non-linearity ϕ**



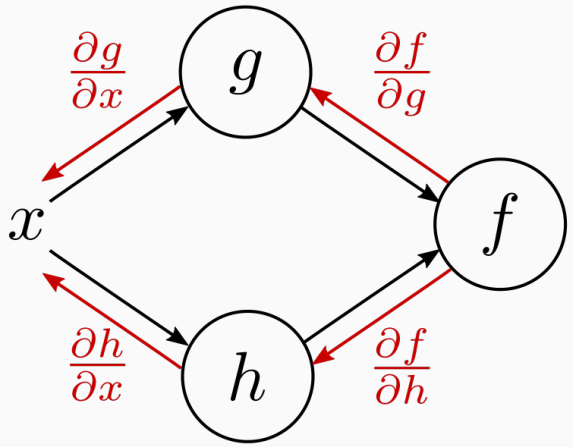
- **shallow networks** (1-layer)
  - **universal approximators** for many non-linearities
  - however, some function classes require **exponentially** many neurons, in the number of inputs
- **deep networks**
  - can often represent represent the same functions with only **polynomial size**
  - this phenomenon is called **expressive efficiency** – Deep networks can represent functions more efficiently than shallow ones

Automatic Differentiation (AD): systematic use of the chain rule

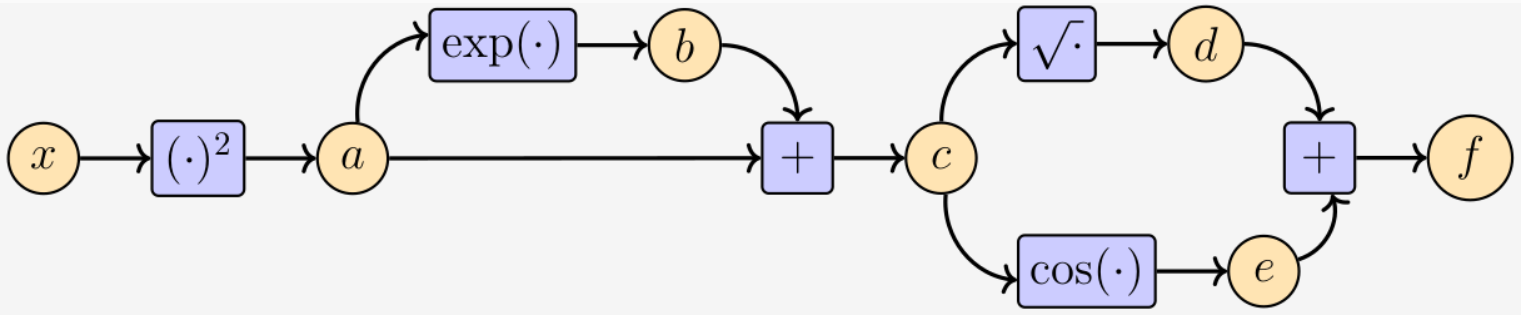
$$\frac{\partial f \circ g}{\partial x} = \frac{\partial f}{\partial g}\frac{\partial g}{\partial x}$$



$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g}\frac{\partial g}{\partial x} + \frac{\partial f}{\partial h}\frac{\partial h}{\partial x}$$

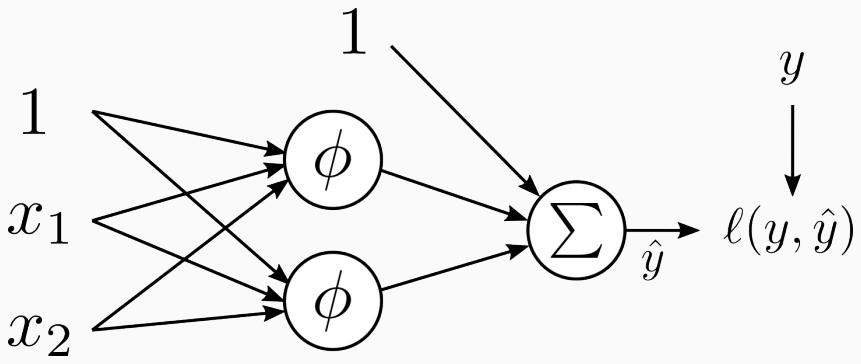


Backwards Pass (Autodiff, Backprop)

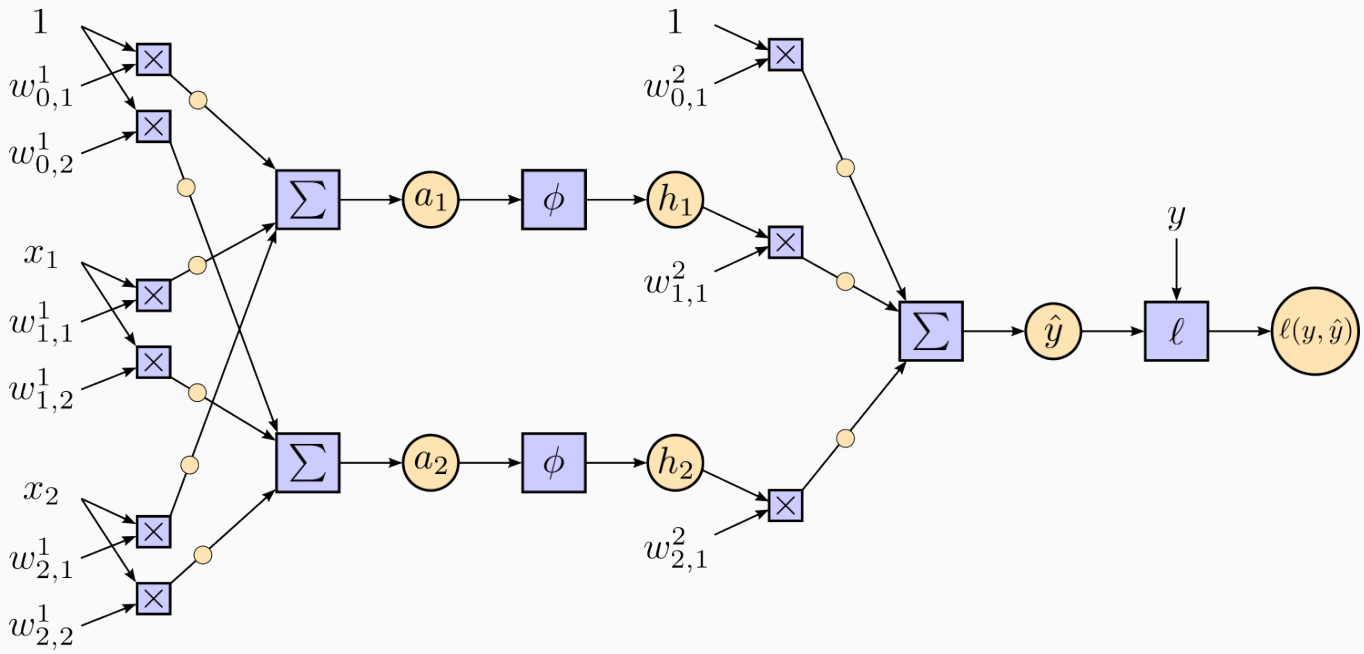


- $\frac{\partial f}{\partial f} = 1$
- $\frac{\partial f}{\partial e} = \frac{\partial f}{\partial f}\frac{\partial f}{\partial e} = 1 \times 1 = 1$
- $\frac{\partial f}{\partial d} = \frac{\partial f}{\partial f}\frac{\partial f}{\partial d} = 1 \times 1 = 1$
- $\frac{\partial f}{\partial c} = \frac{\partial f}{\partial e}\frac{\partial e}{\partial c} + \frac{\partial f}{\partial d}\frac{\partial d}{\partial c} = 1 \times -\sin(c) + 1 \times \frac{1}{2}c^{-\frac{1}{2}} = -0.822$
- $\frac{\partial f}{\partial b} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial b} = -0.822 \times 1 = -0.822$
- $\frac{\partial f}{\partial a} = \frac{\partial f}{\partial c}\frac{\partial c}{\partial a} + \frac{\partial f}{\partial b}\frac{\partial b}{\partial a} = -0.822 \times 1 + -0.822 \times \exp(a) = -45.717$
- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a}\frac{\partial a}{\partial x} = -45.717 \times 2x = -182.87$
- note we get derivatives for **all nodes**, not only x!

- let a computational graph be given
- compute a **forward pass** with given inputs, delivering the output o and all intermediate results
- store all computations "in" their respective nodes
- **reverse-mode autodiff (backprop)** computes for any node n the partial derivative $\frac{\partial o}{\partial n}$ in a **backward-pass**
- **initialization**: **trivial** derivative $\frac{\partial o}{\partial o} = 1$ at output node o
- **recursive principle:** the partial derivatives $\frac{\partial o}{\partial s_i}$ for all **successors** s_i of n have already been computed (stored in s_i)
- for n the partial derivative is $\frac{\partial o}{\partial n} = \sum_i \frac{\partial o}{\partial s_i} \frac{\partial s_i}{\partial n}$
- $\frac{\partial o}{\partial n}$ is stored in n; continue with backward-pass



- one hidden layer with two neurons, sigmoid activation function
- loss function $\ell(y, \hat{y})$, with squared loss (regression problem)
- weights $w^l_{i,j}$ sitting on input edges to each neuron
- parameter vector $\boldsymbol{\theta} = (w^1_{0,1}, w^1_{0,2}, \ldots, w^2_{2,1})^T$
- goal: compute $\nabla_{\boldsymbol{\theta}} \ell = \left(\frac{\partial \ell}{\partial w^1_{0,1}}, \frac{\partial \ell}{\partial w^1_{0,2}}, \ldots, \frac{\partial \ell}{\partial w^2_{2,1}} \right)^T$ for given $\boldsymbol{x}, y$



- the MLP corresponds to the **computational graph** above
- note that weights $w^l_{i,j}$ and target y are also inputs
- the loss ℓ is the output node of the graph
- products are not explicitly stored—instead we consider affine functions as one computation
- we use standard autodiff do compute $\frac{\partial \ell}{\partial w^l_{i,j}}$, for all l, i, j

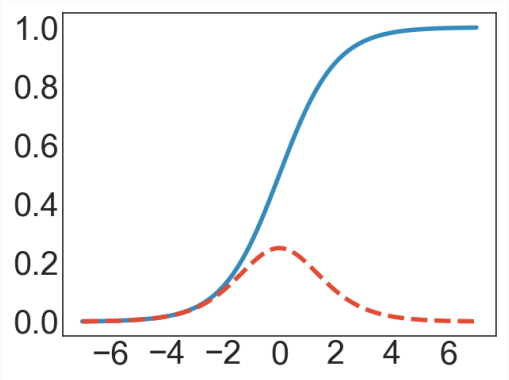
Recap:

- for **squared loss** $\ell(y, \hat{y}) = (\hat{y} - y)^2$ we have

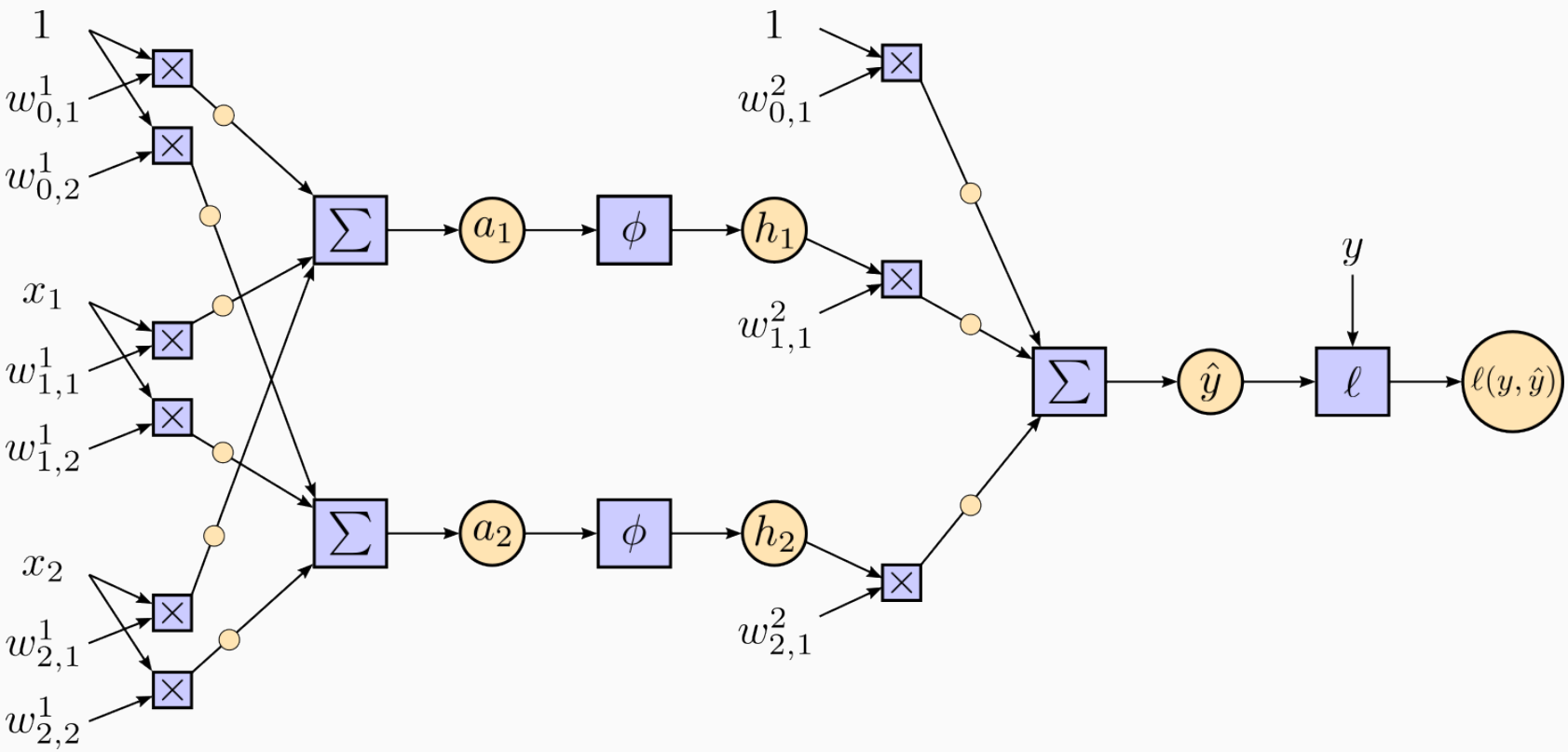
$$\frac{\partial \ell}{\partial \hat{y}} = 2(\hat{y} - y)$$

- for **sigmoid** $\phi(z) = \frac{1}{1+\exp(-z)}$ we have

$$\frac{\partial \phi}{\partial z} = \sigma(z)\sigma(-z) = \sigma(z)(1 - \sigma(z))$$

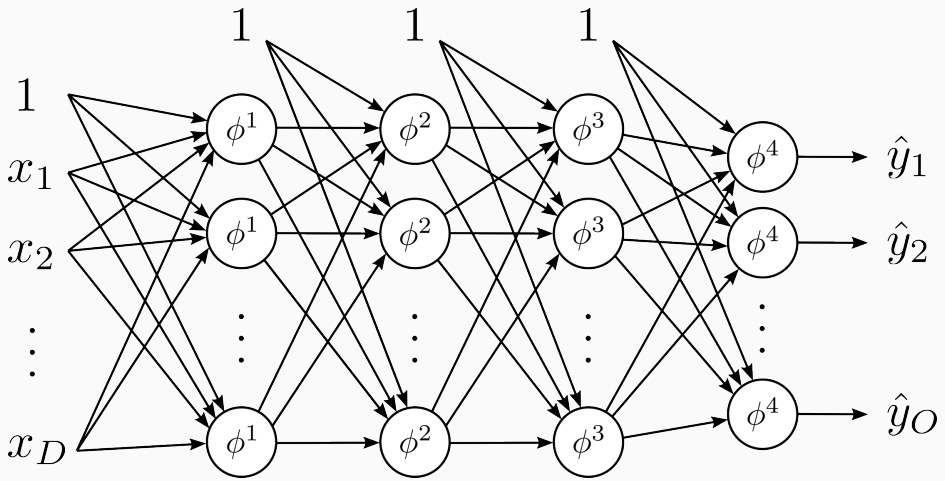


- generally, loss and activation functions in NNs have "easy" analytic derivatives


1
$w^1_{0,1}$
$w^1_{0,2}$
x_1
$w^1_{1,1}$
$w^1_{1,2}$
x_2
$w^1_{2,1}$
$w^1_{2,2}$
$\sum$
a_1
ϕ
h_1
a_2
ϕ
h_2
1
$w^2_{0,1}$
$w^2_{1,1}$
$w^2_{2,1}$
$\sum$
$\hat{y}$
y
ℓ
$\ell(y, \hat{y})$

Matrix-Vector Notation for MLPs

Multi Layer Perceptrons



- $a_j^l = \sum_{k=1}^{H_{l-1}} w_{k,j}^l \, h_k^{l-1}$ is the **activation** of the jth unit in the lth layer
- $h_j^l = \phi^l(a_j^l)$ is the output of the jth unit in the lth layer
- **written much more compactly using matrix-vector notation**
- math becomes actually easier
- vectorized implementations are typically faster, especially on dedicated hardware (GPUs, TPUs, etc.)

Matrix-Vector Notation for MLPs

$$\boldsymbol{h}^0 = (1, x_1, x_2, \ldots, x_D)^\top$$
$$\boldsymbol{h}^l = (1, h_1^l, h_2^l, \ldots, h_{H_l}^l)^\top$$
$$\boldsymbol{a}^l = \left(a_1^l, a_2^l, \ldots, a_{H_l}^l\right)^T$$

$$W^l = \begin{pmatrix} w_{0,1}^l & w_{1,1}^l & \cdots & w_{H_{l-1},1}^l \\ w_{0,2}^l & w_{1,2}^l & \cdots & w_{H_{l-1},2}^l \\ \vdots & \vdots & \ddots & \vdots \\ w_{0,H_l}^l & w_{1,2}^l & \cdots & w_{H_{l-1},H_l}^l \end{pmatrix}$$

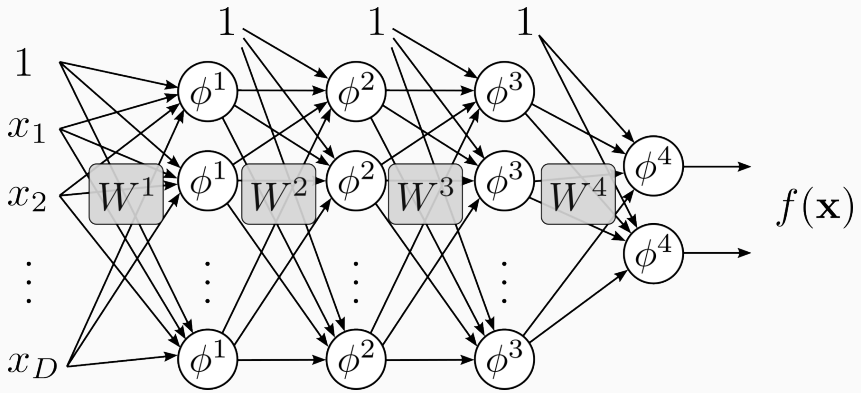
- With this notation, an **activation vector** is given as a matrix-vector product of the previous **output vector** and a **weight matrix**:

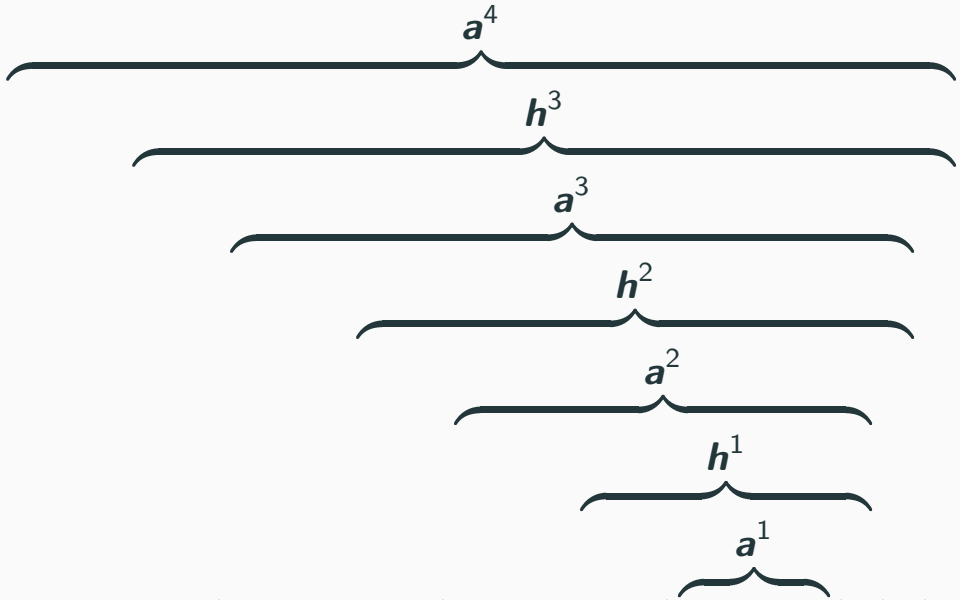
$$\boldsymbol{a}^l = \mathbf{W}^{l\,T}\, \boldsymbol{h}^{l-1}$$

- The outputs vectors are given as

$$\boldsymbol{h}^l = \left(1, \phi^l(a_1^l), \phi^l(a_2^l), \ldots, \phi^l(a_{H_l}^l)\right)^T$$

Writing an MLP in One Line*

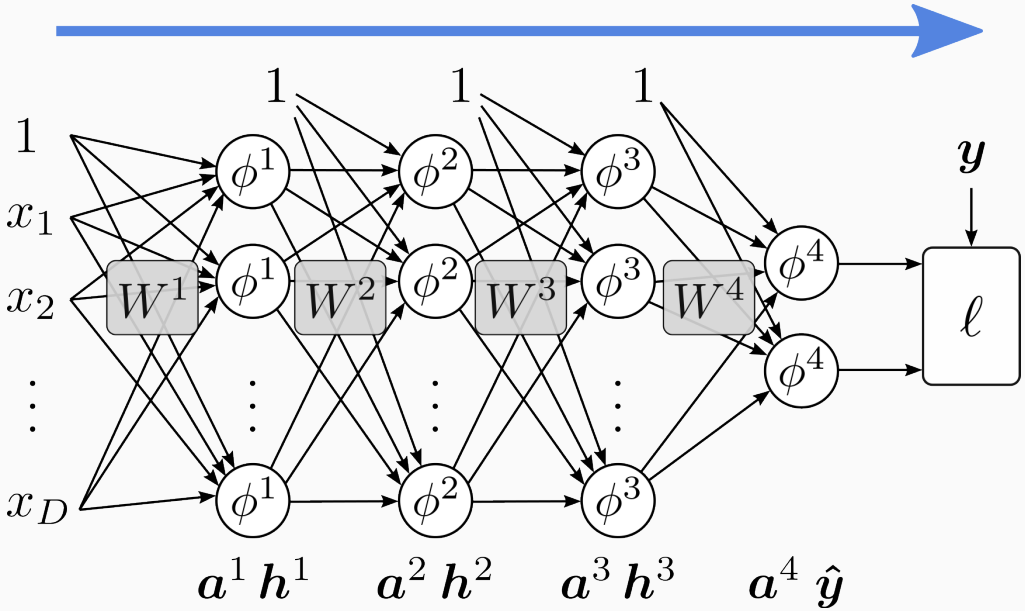




$$f(\boldsymbol{x}) = \phi^4\Big(\mathbf{W}^{4\,T}\,\bar{\phi}^3\Big(\mathbf{W}^{3\,T}\,\bar{\phi}^2\Big(\mathbf{W}^{2\,T}\,\bar{\phi}^1\Big(\overbrace{\mathbf{W}^{1\,T}\boldsymbol{x}}\Big)\Big)\Big)\Big)$$

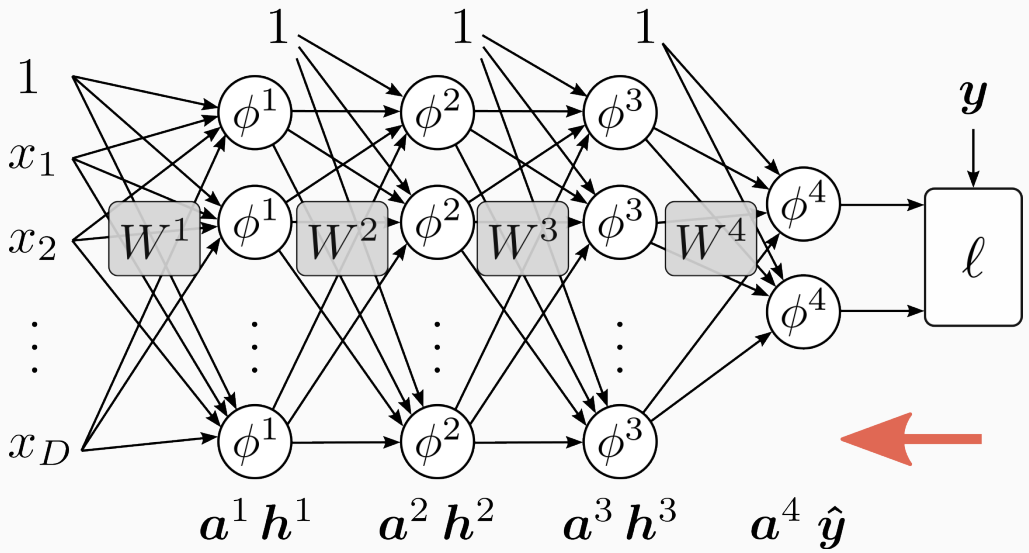
$\bar{\phi}$ denotes adding a "dummy feature" 1.

Autodiff in Matrix-Vector Form



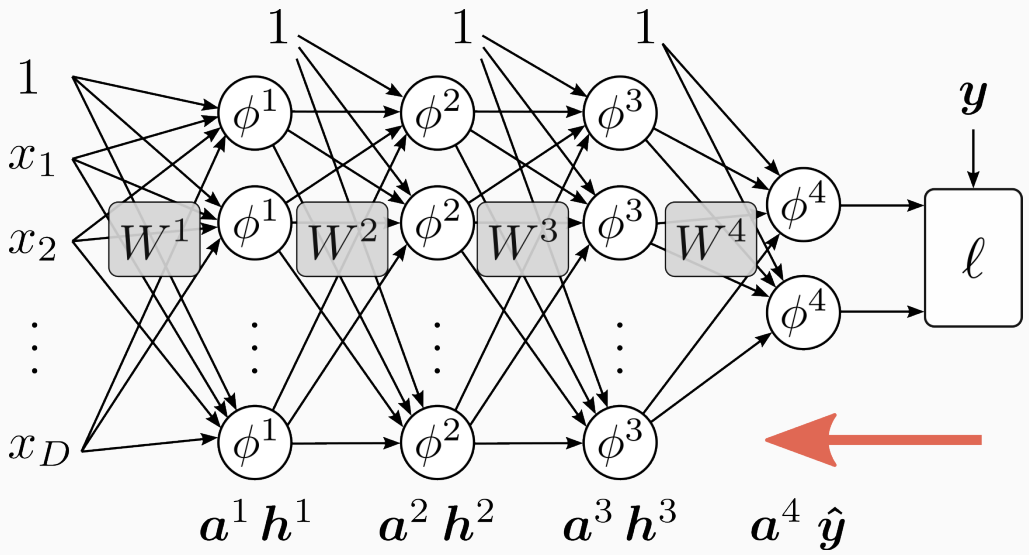
Forward evaluation with input $\boldsymbol{x}$, computing loss $\ell(\hat{\boldsymbol{y}}, \boldsymbol{y})$

Autodiff in Matrix-Vector Form



The gradient of the loss w.r.t. the output layer $\nabla_{\hat{\boldsymbol{y}}^4}\ell = (\frac{\partial \ell}{\partial \hat{y}_1}, \ldots, \frac{\partial \ell}{\partial \hat{y}_O})^T$ is available in closed form for all losses used in practice (e.g., squared loss, cross-entropy)

Autodiff in Matrix-Vector Form

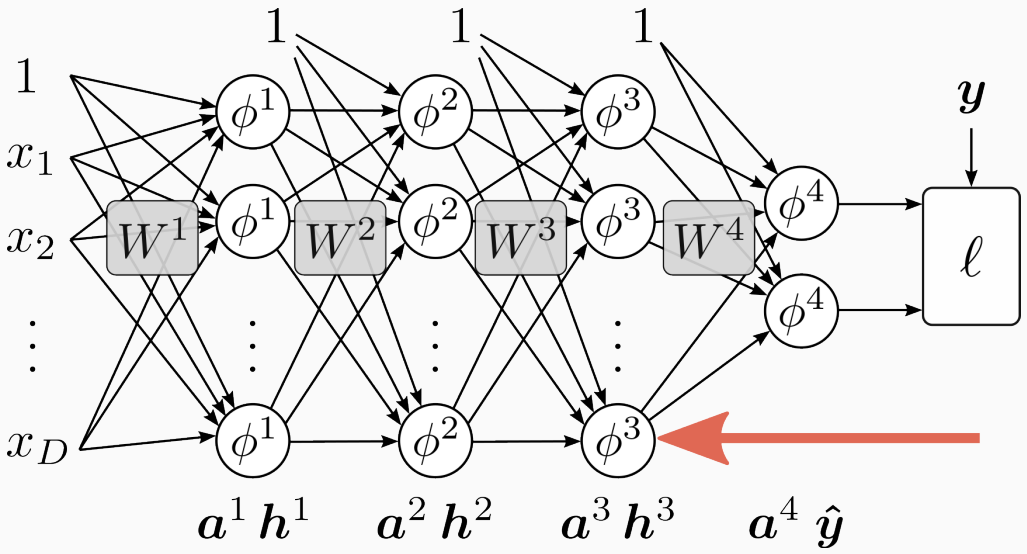


From autodiff we know $\frac{\partial \ell}{\partial a_i^4} = \frac{\partial \ell}{\partial \hat{y}_i} \frac{\partial \hat{y}_i}{\partial a_i^4} = \frac{\partial \ell}{\partial \hat{y}_i} \phi^{4\prime}(a_i^4)$, where $\phi^{4\prime}$ is the derivative of ϕ^4. In short

$$\nabla_{\boldsymbol{a}^4} \ell = \mathit{diag}(\phi^{4\prime}(\boldsymbol{a}^4)) \nabla_{\hat{\boldsymbol{y}}^4} \ell$$

Here $\mathit{diag}(\boldsymbol{z})$ is the diagonal matrix whose diagonal is $\boldsymbol{z}$.

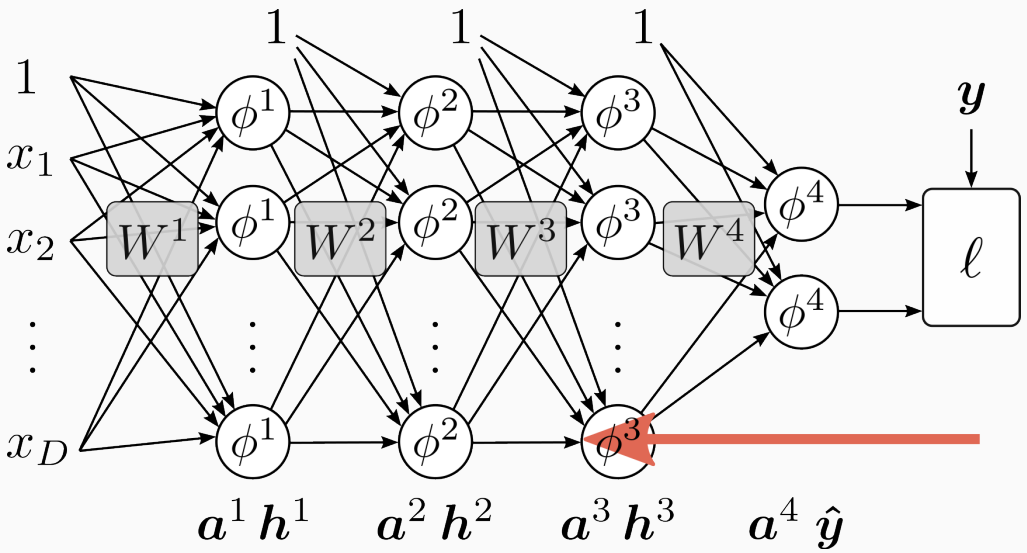
Autodiff in Matrix-Vector Form



From autodiff we know $\frac{\partial \ell}{\partial h_i^3} = \sum_{j=1}^{O} \frac{\partial \ell}{\partial a_j^4} \frac{\partial a_j^4}{\partial h_i^3} = \sum_{j=1}^{O} \frac{\partial \ell}{\partial a_j^4} w_{i,j}^4$ which is an inner product between $\nabla_{\boldsymbol{a}^4} \ell$ and the ith row of W^4. In short

$$\nabla_{\boldsymbol{h}^3} \ell = W^4 \nabla_{\boldsymbol{a}^4} \ell$$

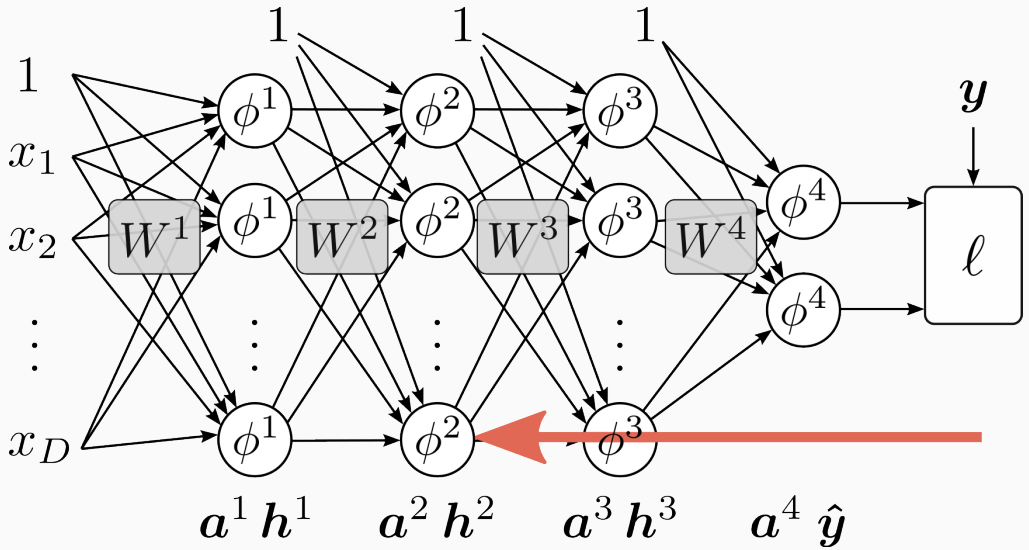
Autodiff in Matrix-Vector Form



and so on . . .

$$\nabla_{\boldsymbol{a}^3}\ell = diag(\phi^{3\prime}(\boldsymbol{a}^3))\,\nabla_{\boldsymbol{h}^3}\ell$$

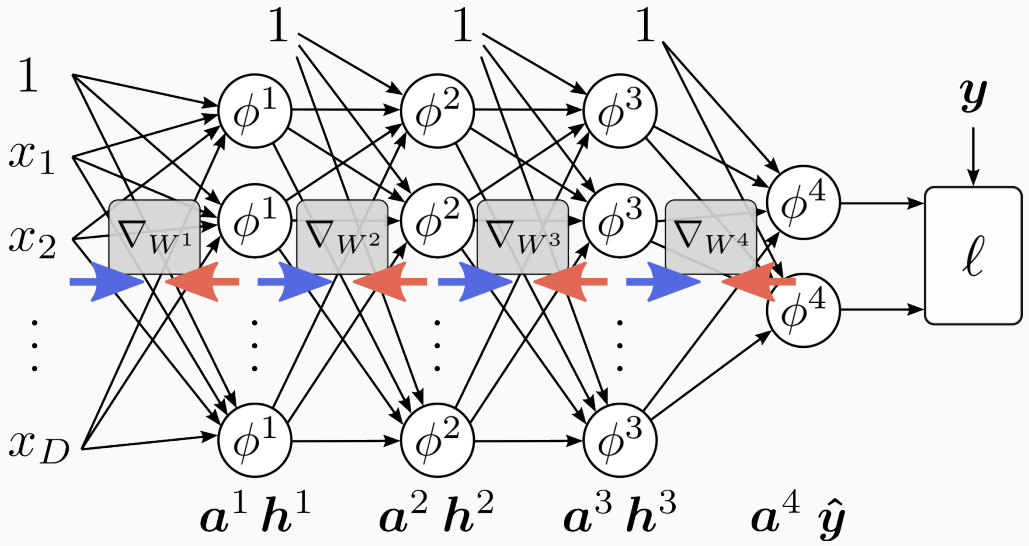
Autodiff in Matrix-Vector Form



and so on . . .

$$\nabla_{\boldsymbol{h}^2} \ell = W^3 \nabla_{\boldsymbol{a}^3} \ell$$

Autodiff in Matrix-Vector Form



With just two rules, we get the derivatives w.r.t. all activations a_i^l and outputs h_i^l.

The derivatives w.r.t. the weights are then given as

$$\frac{\partial \ell}{\partial w_{i,j}^l} = \frac{\partial \ell}{\partial a_j^l} \frac{\partial a_j^l}{\partial w_{i,j}^l} = \overbrace{\frac{\partial \ell}{\partial a_j^l}}^{\text{backprop}} \overbrace{h_i^{l-1}}^{\text{forward eval}}$$

Classification with MLPs

Modeling Multi-class Problems

- recall that in Lecture 3, we discussed **binary classification** for logistic regression
- we used a linear model $f(\boldsymbol{x}) = \boldsymbol{w}^T \phi(\boldsymbol{x})$ (ϕ is a vector of fixed non-linearities) and converted its output to a probability with the sigmoid function:

$$\pi = \sigma\left(\boldsymbol{w}^T \phi(\boldsymbol{x})\right)$$

- π (respectively $1 - \pi$) is interpreted as the model's belief that $y = 1$ (respectively $y = 0$).
- how to generalize this to arbitrary many classes, i.e. to **multi-class** problems?

Modeling Multi-class Problems

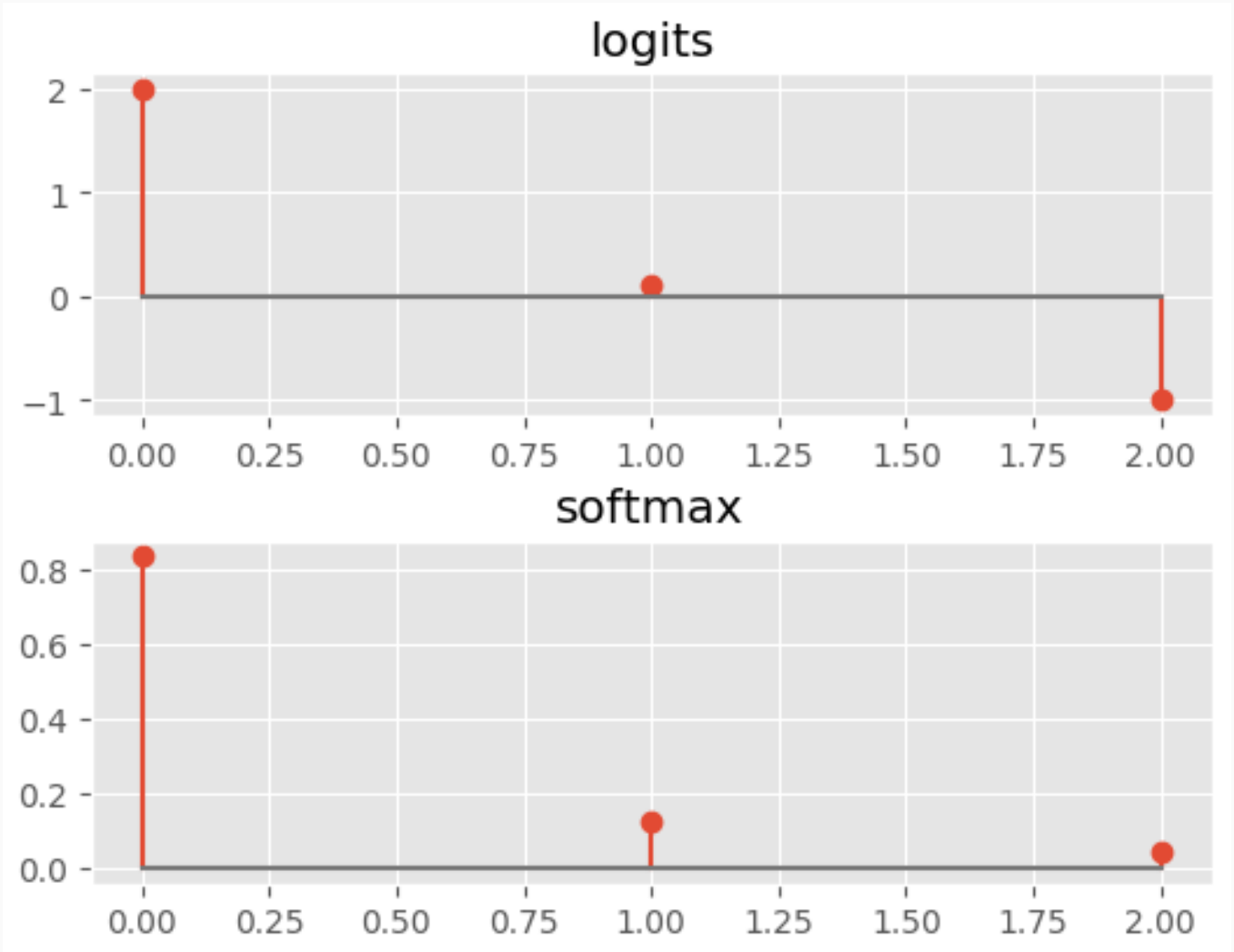
- let C be the number of classes (e.g. $C = 3$ if classes are 'dog', 'cat', 'rabbit').
- introduce one output (neuron) a_c per class and let $\boldsymbol{a} = (a_1, \ldots, a_C)^T$ be the C-dimensional output vector
- Convert $\boldsymbol{a} = (a_1, \ldots, a_C)^T$ into a probability distribution over labels. This is achieved with the so-called **softmax function**:

$$\boldsymbol{\pi} = \begin{pmatrix} \pi_1 \\ \vdots \\ \pi_C \end{pmatrix} = softmax(\boldsymbol{a}) = \begin{pmatrix} \frac{\exp(a_1)}{Z} \\ \vdots \\ \frac{\exp(a_C)}{Z} \end{pmatrix}, \; Z = \sum_{i=1}^{C} \exp(a_i)$$

- evidently, $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_C)^T$ is non-negative and sums to 1, i.e. it is a categorical distribution
- in this context, $\boldsymbol{a}$ are often called the **logits** of the softmax

Softmax Function

Example



- let $\boldsymbol{a}$ be the outputs of the model and $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_C)^T = \mathit{softmax}(\boldsymbol{a})$
- assume that classes are encoded with integers i.e., $y \in \{1, \ldots, C\}$
- then the **cross-entropy** for one sample $\boldsymbol{x}, y$ is defined as

$$\ell_{CE}(\boldsymbol{\pi}, y) = -\log \pi_y$$

  i.e., the negative of the model's log-probability assigned to the true class (negative **log-likelihood**).
- the **cross-entropy loss** over the whole training set is

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^{N} \log \pi_{y^{(i)}}^{(i)}$$

- compute $\nabla_{\boldsymbol{\theta}} \mathcal{L}_{CE}$ with autodiff and perform gradient descent

Gradient of Cross-Entropy

First, we require the gradient of $CE(softmax(\boldsymbol{a}))$.

$$CE = -\log \pi_y \qquad \pi_c = \frac{\exp(a_c)}{\sum_{c'=1}^{C} \exp(a_{c'})}$$

Case 1: $c = y$

$$\frac{\partial CE}{\partial a_c} = -\frac{1}{\pi_y}\left[\overbrace{\frac{\exp(a_c)}{\sum_{c'=1}^{C}\exp(a'_c)}}^{\pi_y} - \overbrace{\frac{\exp^2(a_c)}{\left(\sum_{c'=1}^{C}\exp(a'_c)\right)^2}}^{\pi_y^2}\right] = (\pi_c - 1)$$

Case 2: $c \neq y$

$$\frac{\partial CE}{\partial a_c} = -\frac{1}{\pi_y}\left[-\overbrace{\frac{\exp(a_y)\exp(a_c)}{\left(\sum_{c'=1}^{C}\exp(a'_c)\right)^2}}^{\pi_y\pi_c}\right] = \pi_c$$

After that, just use standard autodiff.

Detailed derivation (not for exam)

(1) derivative of cross-entropy by π_c

$$\frac{\partial CE}{\partial \pi_c} = \begin{cases} -\frac{1}{\pi_c} & \text{if } y = c \\ 0 & \text{otherwise} \end{cases}$$

(2) derivative of softmax

Note that $\frac{\exp(a_c)}{\sum_{c'=1}^{C} \exp(a_{c'})} = \exp(a_c) \left(\sum_{c'=1}^{C} \exp(a_{c'}) \right)^{-1}$

Hence by the product rule

$$\frac{\partial \pi_c}{\partial a_d} = \frac{\partial \exp(a_c)}{\partial a_d} \left(\sum_{c'=1}^{C} \exp(a_{c'}) \right)^{-1} + \exp(a_c) \frac{\partial \left(\sum_{c'=1}^{C} \exp(a_{c'}) \right)^{-1}}{\partial a_d}$$

$$= \frac{\frac{\partial \exp(a_c)}{\partial a_d}}{\sum_{c'=1}^{C} \exp(a_{c'})} - \frac{\exp(a_c)}{\left(\sum_{c'=1}^{C} \exp(a_{c'}) \right)^2} \frac{\partial \sum_{c'=1}^{C} \exp(a_{c'})}{\partial a_d}$$

$$\frac{\frac{\partial \exp(a_c)}{\partial a_d}}{\sum_{c'=1}^{C} \exp(a_{c'})} = \begin{cases} \frac{\exp(a_c)}{\sum_{c'=1}^{C} \exp(a_{c'})} & \text{if } c = d \\ 0 & \text{otherwise} \end{cases}$$

$$-\frac{\exp(a_c)}{\left(\sum_{c'=1}^{C} \exp(a_{c'})\right)^2} \frac{\partial \sum_{c'=1}^{C} \exp(a_{c'})}{\partial a_d} = -\frac{\exp(a_c)\exp(a_d)}{\left(\sum_{c'=1}^{C} \exp(a_{c'})\right)^2}$$

Hence, if $c = d$:

$$\frac{\partial \pi_c}{\partial a_d} = \frac{\exp(a_c)}{\sum_{c'=1}^{C} \exp(a_{c'})} - \frac{\exp^2(a_c)}{\left(\sum_{c'=1}^{C} \exp(a_{c'})\right)^2}$$

and if $c \neq d$:

$$\frac{\partial \pi_c}{\partial a_d} = -\frac{\exp(a_c)\exp(a_d)}{\left(\sum_{c'=1}^{C} \exp(a_{c'})\right)^2}$$

$$\frac{\partial CE}{\partial a_c} = \sum_d \overbrace{\frac{\partial CE}{\partial \pi_d}}^{\text{0, except if } d=y} \frac{\partial \pi_d}{\partial a_c} = -\frac{1}{\pi_y} \frac{\partial \pi_y}{\partial a_c}$$

Case 1: $c = y$

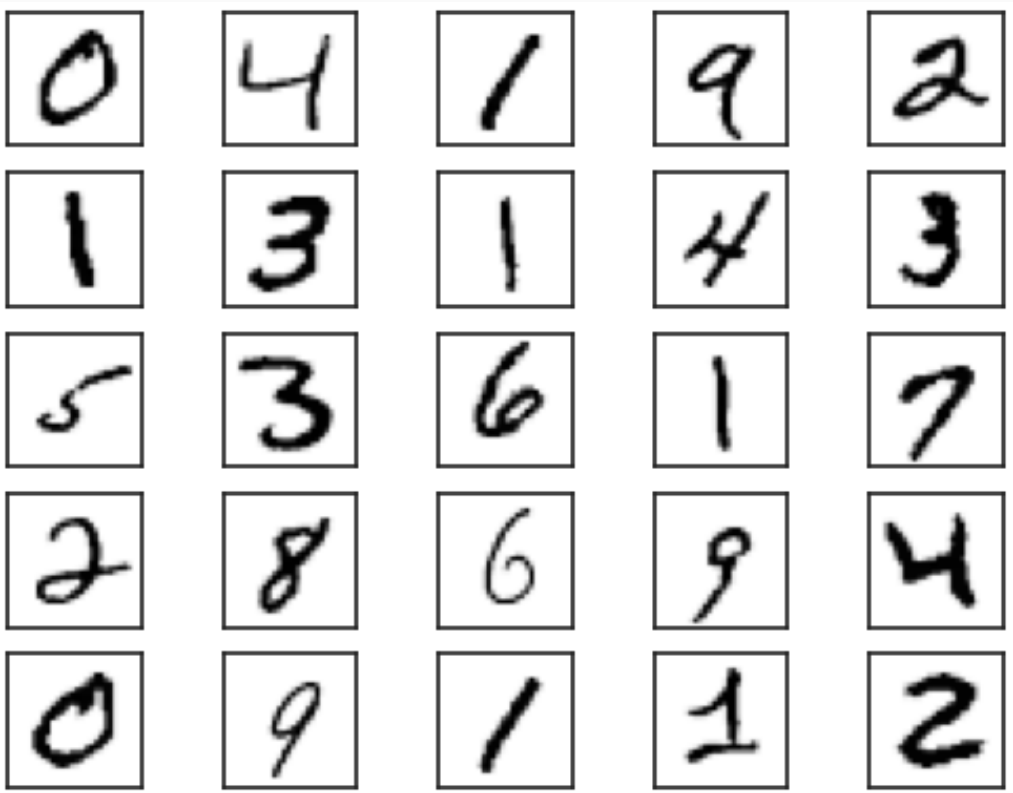
$$\frac{\partial CE}{\partial a_c} = -\frac{1}{\pi_y} \left[\overbrace{\frac{\exp(a_c)}{\sum_{c'=1}^{C} \exp(a'_c)}}^{\pi_y} - \overbrace{\frac{\exp^2(a_c)}{\left(\sum_{c'=1}^{C} \exp(a'_c)\right)^2}}^{\pi_y^2} \right] = (\pi_c - 1)$$

Case 2: $i \neq y$

$$\frac{\partial CE}{\partial a_c} = -\frac{1}{\pi_y} \left[-\overbrace{\frac{\exp(a_y)\exp(a_c)}{\left(\sum_{c'=1}^{C} \exp(a'_c)\right)^2}}^{\pi_y \pi_c} \right] = \pi_c$$

MNIST Digit Classification

- $60k$ training images, $10k$ test images
- split training images into $50k$ training examples and $10k$ validation examples
- 28×28 pixels, yields 784-dimensional feature vectors



MNIST Digit Classification cont'd

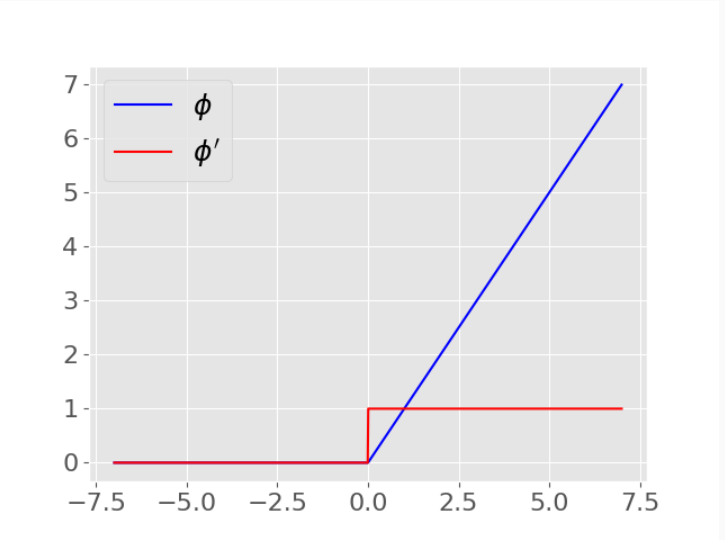
Example

- MLP with two hidden layers, 1000 neurons each
- 10 outputs, one for each class
- ReLU activations for hidden layers
- outputs are processed by softmax
- using cross-entropy loss
- initialize parameters randomly (small uniform noise)
- 500 iterations of gradient descent (often called **epochs**)
- step-size $\eta = 0.1$

Rectified Linear Unit (ReLU)

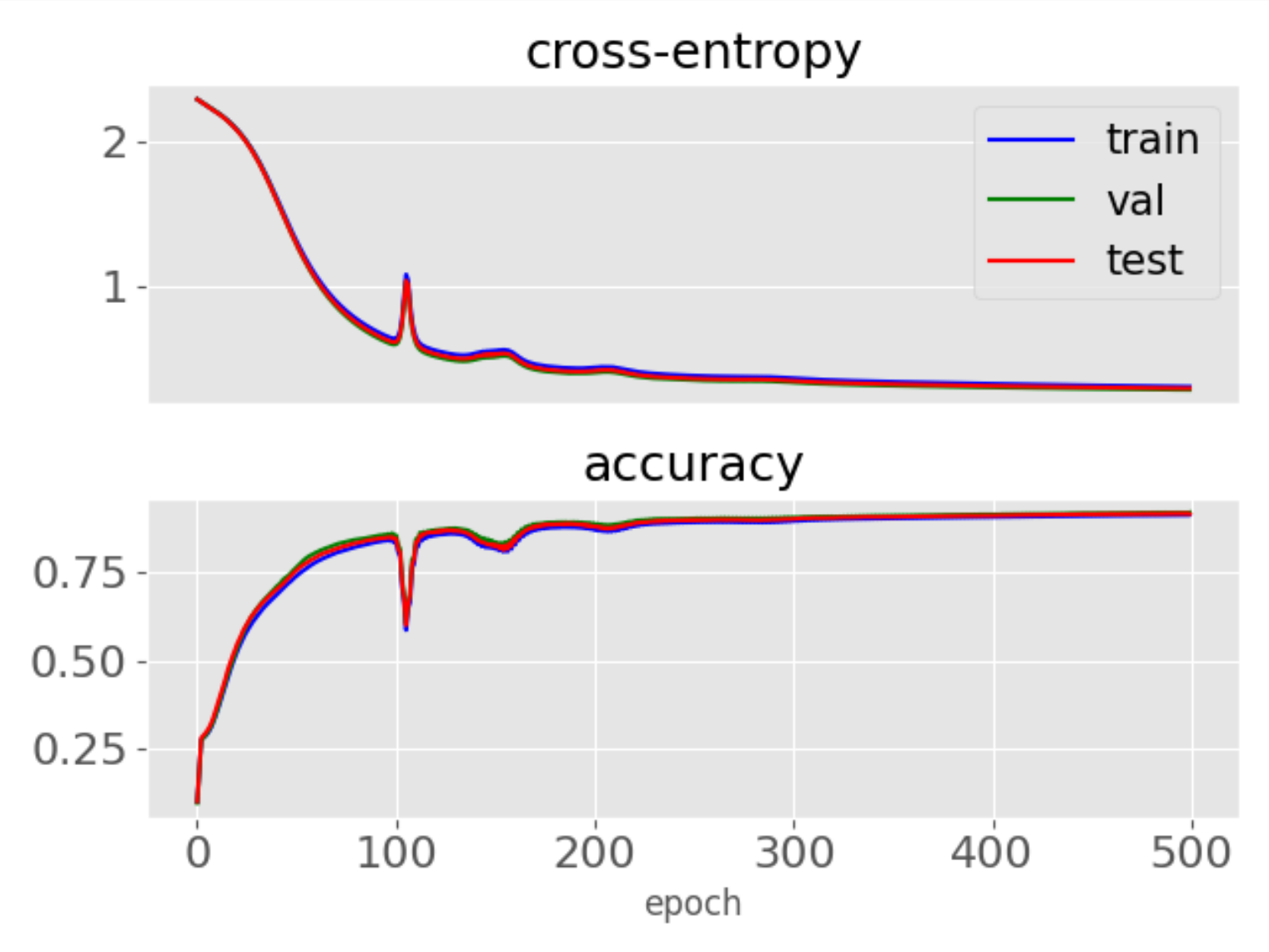
$\phi(x) = \max(x, 0)$

$\phi'(x) = \mathbb{1}(x > 0)$



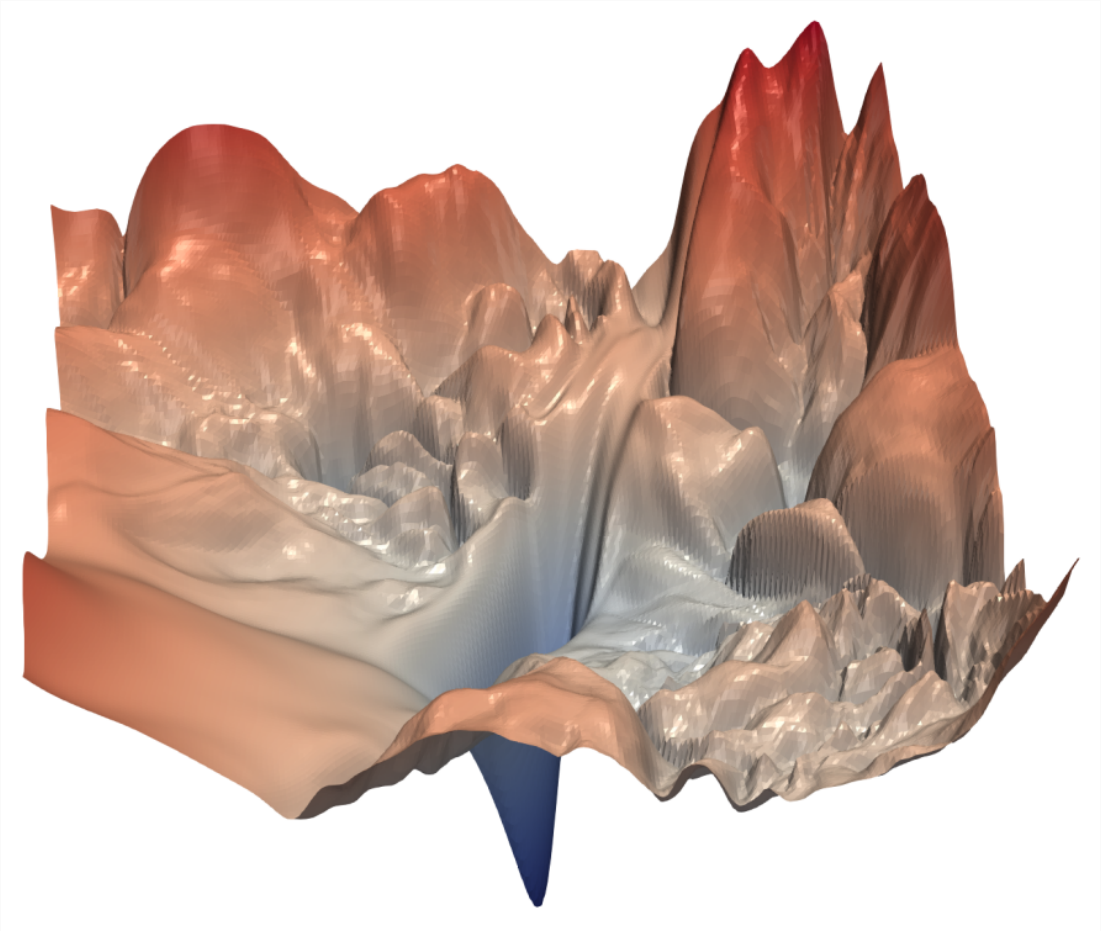
MNIST Digit Classification cont'd

Example



Loss Landscape of Neural Networks (Non-Convex)

Training loss in neural networks is highly non-convex, and we can find only local minima. However, in practice they work very well.



Source: Li et al., *Visualizing the Loss Landscape of Neural Nets*, NeurIPS 2018.

Neural Networks in Practice

Frameworks

- many ML frameworks for training neural nets exist
- **Theano** (2007) was one of the first widely used packages
- todays common ML packages include **Tensorflow**, **PyTorch** and **Jax**
- only forward pass needs to be specified—gradients are obtained via autodiff
- these packages also have various gradient-based optimizers implemented (adaptive versions of standard gradient-descent, like **adam**)
- these frameworks also work well with dedicated hardware, i.e. **Graphics Processing Units (GPUs)**, **Tensor Processing Units (TPUs)**, etc., allowing dramatic speedups

Pre-processing

Input features might live on very different **scales**. For example, assume that some features are measured in *mm* while others are measured in *km*. Thus, good practice is to **normalize** features before using them in neural networks.

The pre-processing is computed from the training set. The same pre-processing, however, needs to be also applied to validation and test sets!

Pre-processing cont'd

Zero-Mean/Unit-Variance Normalization: Let $\bar{\boldsymbol{x}}$ and $\boldsymbol{s}$ be the empirical mean and standard deviation vectors, i.e. $\bar{\boldsymbol{x}} = \frac{1}{N_{train}} \sum_{i=1}^{N_{train}} \boldsymbol{x}^{(i)}$ and $\boldsymbol{s} = \sqrt{\frac{1}{N_{train}} \sum_{i=1}^{N_{train}} (\boldsymbol{x}^{(i)} - \bar{\boldsymbol{x}})^2}$ (here, the square acts element-wise). Then the zero-mean, unit-variance normalization is given as:

$$\boldsymbol{x}^{(i)} \leftarrow \frac{\boldsymbol{x}^{(i)} - \bar{\boldsymbol{x}}}{\boldsymbol{s}}$$

Here the division acts element-wise. The transformed data has zero mean and unit standard deviation (unit variance) in each dimension.

Pre-processing cont'd

Min-Max Normalization: Let $\boldsymbol{x}_{min}$ and $\boldsymbol{x}_{max}$ be the vectors containing minima and maxima over the training set for each dimension, i.e.

- $x_{min,d} = \min_{i=1\ldots N_{train}} x_d^{(i)}$
- $x_{max,d} = \max_{i=1\ldots N_{train}} x_d^{(i)}$

Then the min-max is given as:

$$\boldsymbol{x}^{(i)} \leftarrow \frac{\boldsymbol{x}^{(i)} - \boldsymbol{x}_{min}}{\boldsymbol{x}_{max} - \boldsymbol{x}_{min}}$$

Here the division acts element-wise. The transformed data is scaled to interval $[0, 1]$ in each dimension. Additionally, one might subtract 0.5, so that the data is scaled to interval $[-0.5, 0.5]$.

Stochastic Gradient Descent – Motivation

- gradient of training loss

$$\nabla_{\theta} \mathcal{L}_{train} = \frac{1}{N} \sum_{i=1}^{N} \nabla_{\theta} \ell(y^{(i)}, \hat{y}^{(i)})$$

  requires a whole pass over train data (epoch), **for each single gradient update**

- inefficient, due to **data redundancies**
- extreme example of data redundancy: train data duplicated 10 times yields same gradient $\Rightarrow$ wasted computation
- redundancies in real data are not as extreme, but still lead to wasteful computation
- **let's do quicker updates**

- at the beginning of each epoch, shuffle training samples and divide them into **mini-batches** of size B
- in each epoch, iterate over mini-batches
  - compute loss $\mathcal{L}_{batch} = \frac{1}{B} \sum_{i=1}^{B} \ell(y^{(i)}, \hat{y}^{(i)})$ for current mini-batch
  - update $\boldsymbol{\theta}$ with gradient descent step:

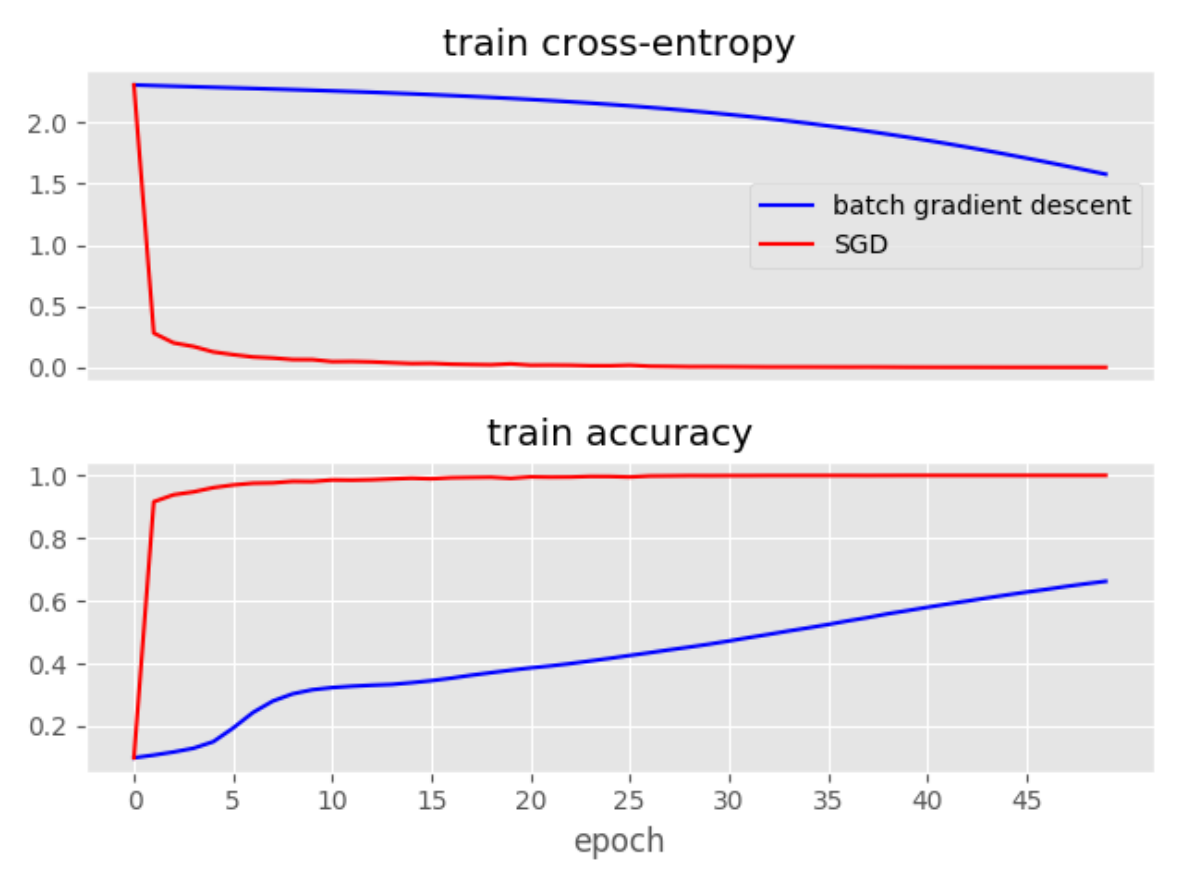
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta_k \nabla_{\boldsymbol{\theta}} \mathcal{L}_{batch}$$

SGD is a Sound Optimizer

- noisy gradients, but **unbiased** (correct expectation, they are the correct gradient "on average")
- if $\sum_k \eta_k = \infty$ and $\sum_k \eta_k^2 < \infty$, where k is the epoch counter, SGD converges to a local minimum of $\mathcal{L}_{train}$! (*)
- e.g. $\eta_k = \frac{1}{k}$ fulfils these requirements
- in contrast to SGD, standard gradient descent is often called batch gradient descent ((!) don't confuse with mini-batches)

*In practice, one often uses a simple stepsize schedule

- MNIST data
- MLP with 2 hidden layers, 200 units each, ReLU activation
- batch size $B = 100$ (600 batches per epoch)
- stepsize $\eta = 0.1$

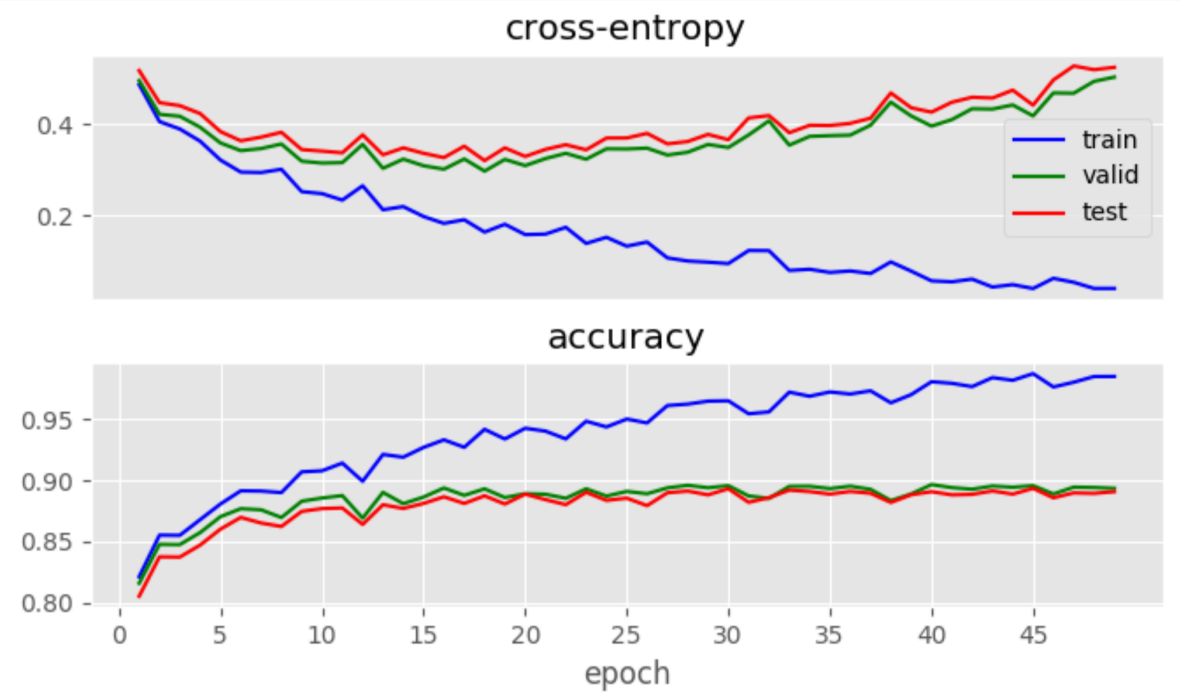


Avoiding Overfitting

Neural networks can have millions and billions of parameters and are thus prone to overfitting. Some common techniques to avoid overfitting are

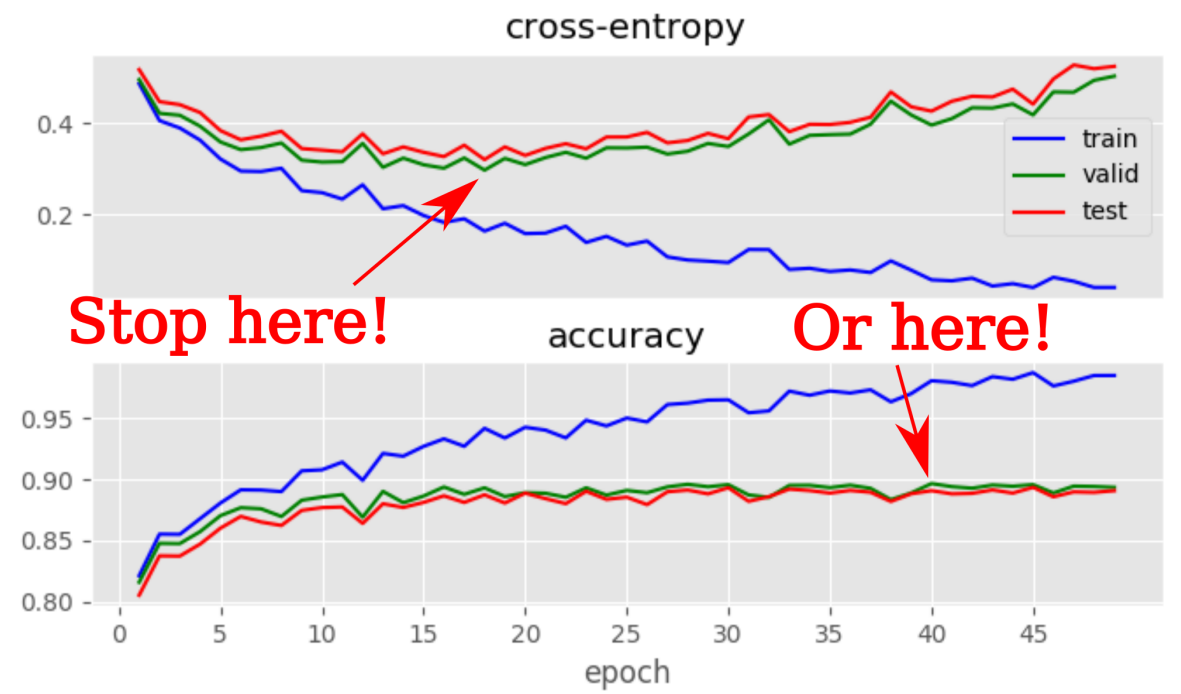
- **Early stopping**
- **Regularization**
- **Dropout**

Early Stopping



- stop, when validation error starts to increase
- epoch number as "hyper parameter"
- save **checkpoint**, whenever observing new minimum $\mathcal{L}_{val,min}$
- optionally: stop training when K_{fail} epochs have passed without decreasing $\mathcal{L}_{val,min}$

Early Stopping



- stop, when validation error starts to increase
- epoch number as "hyper parameter"
- save **checkpoint**, whenever observing new minimum $\mathcal{L}_{val,min}$
- optionally: stop training when K_{fail} epochs have passed without decreasing $\mathcal{L}_{val,min}$

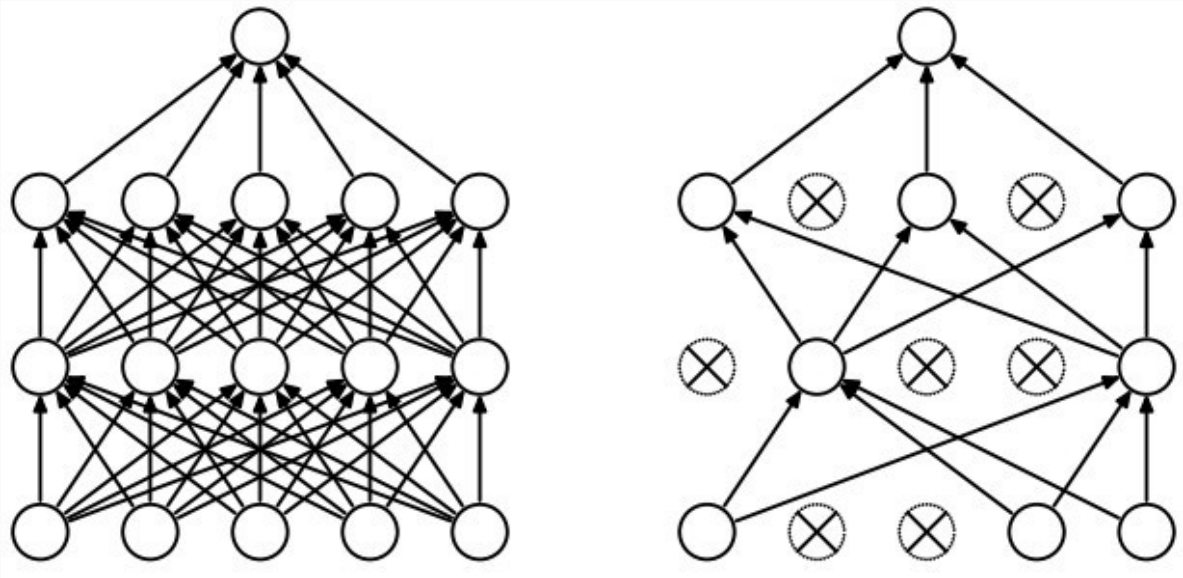
Regularization

- neural nets with smaller weights yield "simpler functions"
- add weight regularizer to loss

$$\mathcal{L}_{train}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \ell(y^{(i)}, f(\boldsymbol{x}^{(i)}; \boldsymbol{\theta})) + \lambda R(\boldsymbol{\theta})$$

- **trade-off parameter** λ balances between data fit and simplicity
- λ can be set via cross-validation
- typical regularization terms:
  - $L2$-**regularizer** $R = \|\boldsymbol{\theta}\|_2^2$
  - $L1$-**regularizer** $R = \|\boldsymbol{\theta}\|_1$
- $L1$-regularizer induces **sparsity**, i.e. many weights will be exactly zero and can be pruned

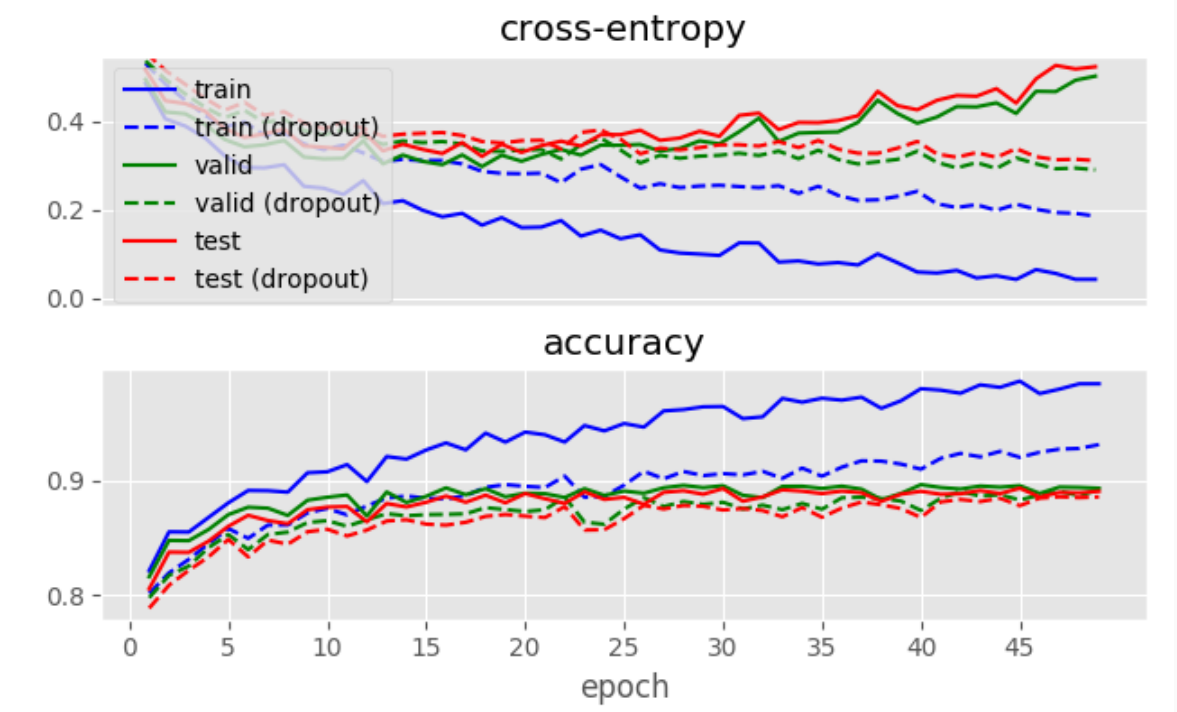
Dropout [Srivastava et al., JMLR 2014]



- randomly drop out inputs and/or hidden units
- avoids "co-adaption" of units
- can be interpreted as implicit ensemble
- dropout probabilities are hyperparameters
- during testing, no dropout is applied, but outputs are scaled by dropout probability to compensate for the additional input



- fashion-MNIST
- train/validation/test split: 50k/10k/10k
- 3-layer MLP with 1000 hidden units each
- SGD, stepsize 0.1, 50 epochs
- dropout at inputs $p = 0.5$



Summary

- autodiff dramatically simplifies learning neural nets
- stochastic gradient descent for quicker updates
- regularization, early stopping, dropout
- see further: **Deep Learning** course on Master level
  - specialized architectures (convolutional neural nets, recurrent neural nets, transformers, . . .)
  - advanced learning techniques
  - advanced applications

Preis für exzellente Lehre



Kontakt

Koordinationsteam Preis für exzellente Lehre

Manuela BERNER
Tel.: +43 316 873 6004

Anna-Maria HENÖGL
Tel.: +43 316 873 6067

Hannes MÜLLER
Tel.: +43 316 873 8553

Katharina SALICITES
Tel.: +43 316 873 8548

lehrpreis@tugraz.at

Datenschutzerklärung

Nominations until 21st of May

lehrpreis@tugraz.at